

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Информационная безопасность

Работа №1

«Разработка защищенного REST API с интеграцией в CI/CD»

Выполнил: студент группы Р3408,
Васильев Никита Алексеевич

Ссылка на GitHub-репозиторий: <https://github.com/kihort-si/ib-1>

Текст файла README.md

```
# Информационная безопасность
## Лабораторная работа №1
### Стек
```

- Java 17 и Maven;
- SQLite и JDBC;
- bcrypt (`jBCrypt`) для хэширования паролей;
- JWT (`java-jwt`) для аутентификации;
- OWASP Java Encoder для экранирования пользовательских данных;
- SpotBugs для SAST;
- OWASP Dependency-Check для SCA.

Архитектура

```
```text
src/main/java/ru/itmo
├── App.java # точка входа
├── config/ # настройки, сборка приложения и БД
├── controller/ # обработчики HTTP-запросов
├── dto/ # модели запросов и ответов API
├── exception/ # прикладные исключения
├── handler/ # единая обработка ошибок
├── middleware/ # JWT-проверка и security-заголовки
├── model/ # доменные модели
├── repository/ # интерфейс и SQLite-реализация доступа к данным
├── routing/ # маршруты приложения
├── security/ # JWT и безопасное кодирование вывода
└── service/ # бизнес-логика регистрации и входа
```
```

API

Все тела запросов и ответов передаются в JSON. Для POST-запросов требуется заголовок `Content-Type: application/json`.

`POST /auth/register`

Создаёт пользователя. Логин должен содержать 3-32 символа из набора `A-Z`, `a-z`, `0-9`, `.` , `\_`, `-`. Пароль должен содержать 12-72 символа, хотя бы одну прописную букву, одну строчную букву и одну цифру.

Запрос:

```
```bash
curl -i -X POST http://localhost:7070/auth/register \
 -H 'Content-Type: application/json' \
 -d '{"login":"student","password":"StrongPass123"}'
```
```

Успешный ответ — `201 Created`:

```
```json
{
 "message": "User registered",
 "login": "student"
}
```

Возможные ошибки: `400 Bad Request` при неверном формате данных и `409`

Conflict`, если логин уже занят.

```
`POST /auth/login`
```

Проверяет логин и пароль и выдаёт JWT со сроком действия 15 минут.

Запрос:

```
```bash
curl -i -X POST http://localhost:7070/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"login":"student","password":"StrongPass123"}'
```
```

Успешный ответ — `200 OK`:

```
```json
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "tokenType": "Bearer",
  "expiresIn": 900
}
```
```

```
`GET /api/data`
```

Возвращает демонстрационный список данных. Это защищённый эндпоинт: JWT нужно передать по схеме Bearer.

```
```bash
curl -i http://localhost:7070/api/data \
  -H "Authorization: Bearer $TOKEN"
```
```

Успешный ответ — `200 OK`:

```
```json
{
  "requestedBy": "student",
  "items": [
    {"id": 1, "title": "Parameterized SQL queries"},
    {"id": 2, "title": "JWT authentication"},
    {"id": 3, "title": "Escaped API output"}
  ]
}
```
```

```
Реализованные меры защиты
```

```
Защита от SQL-инъекций
```

- Любой пользовательский ввод передаётся в SQLite только через `PreparedStatement` и параметры `?`.
- Значения устанавливаются методами `setString`; SQL-запросы не собираются конкатенацией строк.
- Работа с БД сосредоточена в `SqliteUserRepository`, поэтому контроллеры и сервисы не выполняют SQL напрямую.
- Структура БД создаётся статическим запросом без пользовательских значений.
- Интеграционный тест отправляет логин `' OR 1=1 --` и проверяет, что аутентификация завершается ответом `401`, а не обходится.

Пример безопасного запроса из репозитория:

```
```java
```

```
private static final String SELECT_BY_LOGIN =
    "SELECT login, password_hash FROM users WHERE login = ?";

PreparedStatement statement = connection.prepareStatement(SELECT_BY_LOGIN);
statement.setString(1, login);
```

```

#### #### Защита от XSS

- API всегда возвращает JSON, а не формирует HTML из пользовательского ввода.
- Значения, полученные от пользователя и возвращаемые в ответе (`login`, имя аутентифицированного пользователя и сообщения ошибок), проходят контекстное экранирование `Encode.forHtmlContent` из OWASP Java Encoder.
- Формат логина ограничен безопасным allowlist-регулярным выражением и не допускает `<`, `>`, кавычки и пробельные символы.
- Каждый ответ получает заголовки `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'` и `X-Content-Type-Options: nosniff`.
- Клиентскому приложению всё равно следует вставлять строки как текст (`textContent`), а не как HTML (`innerHTML`): защита API дополняет, но не заменяет безопасный рендеринг на клиенте.

#### #### Защита аутентификации

- Пароли никогда не сохраняются в открытом виде: перед записью они хэшируются bcrypt с cost factor `12` и индивидуальной солью.
- База данных содержит только логин и bcrypt-хэш. Интеграционный тест отдельно проверяет, что сохранённое значение отличается от пароля и имеет формат bcrypt.
- Для существующего и несуществующего пользователя используется одинаковое сообщение `Invalid login or password`.
- При неизвестном логине выполняется проверка фиктивного bcrypt-хэша, что уменьшает различие во времени ответа и затрудняет перебор имён пользователей.
- После успешного входа выдаётся JWT, подписанный `HMAC-SHA-256`. В токен включены `iss=secure-api`, `sub=<login>`, время выпуска и срок истечения 900 секунд.
- Секрет подписи читается из `JWT\_SECRET` и валидируется: длина должна быть не меньше 32 символов. Секрет не хранится в исходном коде.
- Middleware применяется ко всем маршрутам `/api/\*`, извлекает заголовок `Authorization: Bearer ...`, проверяет подпись, issuer и срок действия токена. Обработчик контроллера запускается только после успешной проверки.
- Ответы содержат `Cache-Control: no-store`, чтобы токены и приватные данные не сохранялись промежуточными кэшами.

#### ## Отчёты SAST и SCA

Проверки автоматически запускаются в GitHub Actions при каждом `push` и создании `pull request`.

kihort-si / lib-1

CodeIssuesPull requestsAgentsActionsProjectsWikiSecurity and qualityInsightsSettings

Security Scan

add curl commands for user authentication and data access testing #9

Re-run all jobs

Summary

All jobs

Tests, SAST and SCA

Run details

Usage

Workflow file

Triggered via push 3 minutes ago

kihort-si pushed · 7d33463

Status

Success

Total duration

1m 34s

Artifacts

2

ci.yml

on: push

Tests, SAST and SCA 1m 29s

Artifacts

Produced during runtime

| Name                     | Size    | Digest                                                  |
|--------------------------|---------|---------------------------------------------------------|
| dependency-check-reports | 76.1 KB | sha256:e25635dffe5e1cac41f29ee1ceb38fc843892d6aaf8ca... |
| spotbugs-report          | 2.49 KB | sha256:9855a77d5c01de0839c226e5d69586851e98a8f139e01... |

### SpotBugs (SAST)

Отчёт статического анализа: предупреждений не обнаружено.

---

# SpotBugs Report

## Project Information

Project: Secure educational API

SpotBugs version: 4.10.4

Code analyzed:

- /home/runner/work/ib-1/ib-1/target/classes

## Metrics

366 lines of code analyzed, in 24 classes, in 12 packages.

| Metric                   | Total    | Density*    |
|--------------------------|----------|-------------|
| High Priority Warnings   |          | 0.00        |
| Medium Priority Warnings |          | 0.00        |
| <b>Total Warnings</b>    | <b>0</b> | <b>0.00</b> |

(\* Defects per Thousand lines of non-commenting source statements)

## Contents

- [Details](#)

## Summary

| Warning Type | Number |
|--------------|--------|
| Total        | 0      |

## Warnings

Click on a warning row to see full context information.

## Details

## ### OWASP Dependency-Check (SCA)

Отчёт проверки зависимостей: известных уязвимостей не обнаружено.



Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

[How to read the report](#) | [Suppressing false positives](#) | [Getting Help: github issues](#)

### Project: Secure educational API

ru.itmo:secure-api:1.0.0

Scan Information ([show all](#)):

- *dependency-check version:* 12.2.2
- *Report Generated On:* Mon, 14 Sep 2026 14:21:23 +0300
- *Dependencies Scanned:* 48 (28 unique)
- *Vulnerable Dependencies:* 0
- *Vulnerabilities Found:* 0
- *Vulnerabilities Suppressed:* 0
- ...

### Summary

Summary of Vulnerable Dependencies ([click to show all](#))

| Dependency | Vulnerability IDs | Package | Highest Severity | CVE Count | Confidence | Evidence Count |
|------------|-------------------|---------|------------------|-----------|------------|----------------|
|------------|-------------------|---------|------------------|-----------|------------|----------------|

### Dependencies (vulnerable)

This report contains data retrieved from the [National Vulnerability Database](#).  
This report may contain data retrieved from the [CISA Known Exploited Vulnerability Catalog](#).  
This report may contain data retrieved from the [Github Advisory Database \(via NPM Audit API\)](#).  
This report may contain data retrieved from the [OSV \(Open Source Vulnerabilities\)](#) database.

Ссылка на последний успешный запуск pipeline в репозитории: <https://github.com/kihorts/ib-1/actions/runs/36870628094/job/110397152764>